

# Mini Tutorial Docker Básico (Completo y Visual)

## 1. Comprueba que Docker está instalado y funcionando:

```
docker --version
docker ps
docker ps -a
```

## 2. Crear y ejecutar contenedores simples (sin mapear puertos) (docker hub)

Ejemplo: Ubuntu interactivo

```
docker run -it --name miubuntu_asir ubuntu:latest bash
```

-it → modo interactivo  
--name miubuntu → nombre del contenedor  
ubuntu:latest → imagen base, versión más reciente  
bash → comando que se ejecuta al entrar

Salir del contenedor:

Sin detenerlo: Ctrl + P + Q

Salir y detener: exit

```
(docker exec -it ubuntu_asir bash)
```

## 3. Ver contenedores

Contenedores en ejecución:

```
docker ps
```

Todos los contenedores (incluidos los detenidos):

```
docker ps -a
```

## 4. Volver a ejecutar un contenedor detenido

- Docker inicia el contenedor en segundo plano (detached) sin adjuntar ni la entrada ni la salida.

```
docker start ubuntu_asir
```

Para interactuar con él desde tu terminal.

```
docker exec -it ubuntu_asir bash
```

- Inicia el contenedor miubuntu y adjunta su salida/interfaz interactiva a la terminal

```
docker start -ai ubuntu_asir
```

-a → adjunta salida

-i → interactivo

*Si solo pones -i en: docker start -i miubuntu*

*Inicia el contenedor y mantiene la entrada estándar (stdin) abierta, pero no adjunta la salida a tu terminal. En otras palabras, puedes enviarle comandos al contenedor, pero no verás la salida directamente en tu terminal.*

- Parar un contenedor

```
docker stop 426bcee283d1
```

```
docker stop ubuntu_asir
```

## 5. Borrar contenedores e imágenes

- Borrar un contenedor detenido:  
`docker rm ubuntu_asir`
- Forzar el Borrado de un contenedor no detenido  
`docker rm -f ubuntu_asir`
- Borrar todos los contenedores detenidos:  
`docker container prune` (no ejecutar)
- Ver imágenes  
`docker images`
- Borrar imagen (no ejecutar)  
`docker rmi ubuntu:22.04`  
`docker rmi ubuntu:latest`
- Borrar todo lo que no se usa ( cuidado, incluye imágenes y volúmenes no referenciados):  
`docker system prune -a` (no ejecutar)

## 6. Acceso desde fuera del contenedor

- Comando

```
docker run -d --name miweb nginx:latest
```

-d significa "detached mode" (modo desacoplado o en segundo plano).

Normalmente, si ejecutas un contenedor sin -d, Docker muestra en la terminal los logs y la salida del proceso que corre dentro del contenedor (por ejemplo, el servidor Nginx).

Con -d, en cambio:

- El contenedor se ejecuta en segundo plano.

- Docker no muestra la salida en pantalla.

- En su lugar, devuelve el ID del contenedor.

- El contenedor sigue corriendo hasta que lo detengas manualmente.

Devuelve algo como:

```
7f8a9b2c3d4e5f6g7h8i9j0k (el ID del contenedor)
```

- Cómo comprobar que está corriendo

```
docker ps
```

| CONTAINER ID | IMAGE        | COMMAND                  | STATUS        | NAMES |
|--------------|--------------|--------------------------|---------------|-------|
| 7f8a9b2c3d4e | nginx:latest | "/docker-entrypoint...." | Up 10 seconds | miweb |

- Cómo detenerlo o verlo

```
docker stop miweb
```

- Ver sus logs (aunque esté en segundo plano):

```
docker logs miweb
```

- Volver a entrar en él (si quieres interactuar):

```
docker exec -it miweb bash
```

- Comando con mapeo de puertos

```
docker run -d --name miweb_1 -p 8080:80 nginx:latest
```

| Parte        | Significado                                                                                             |
|--------------|---------------------------------------------------------------------------------------------------------|
| docker run   | Crea y ejecuta un contenedor                                                                            |
| -d           | Lo ejecuta en segundo plano ( <i>detached mode</i> )                                                    |
| --name miweb | Le da el nombre "miweb" al contenedor                                                                   |
| -p 8080:80   | Conecta el puerto <b>8080</b> de tu PC al puerto <b>80</b> del contenedor (el puerto donde corre Nginx) |
| nginx:latest | Usa la imagen oficial más reciente de Nginx                                                             |

- Abrir en el navegador. Después de ejecutarlo, abre tu navegador y entra en:

```
http://localhost:8080
```

Ejercicio: vamos a hacer que tu contenedor Nginx muestre tu propia página web en lugar de la página por defecto.

- Localiza o crea tu index

```
<!DOCTYPE html>
<html>
<head>
  <title>Mi Web con Nginx y Docker</title>
</head>
<body>
  <h1>¡Hola desde mi contenedor Nginx! </h1>
</body>
</html>
```

- Copia tu archivo index.html directamente dentro del contenedor ya creado

```
docker cp C:\Users\santi\index.html miweb:/usr/share/nginx/html/index.html
```

docker cp = copia archivos entre tu PC y el contenedor.

C:\Users\santi\index.html = ruta del archivo en tu máquina.

miweb:/usr/share/nginx/html/ = carpeta dentro del contenedor donde Nginx busca sus páginas.

Si el contenedor está corriendo, Nginx servirá automáticamente el nuevo index.html (no hace falta reiniciarlo).

Pero si quieres estar seguro:

```
docker restart miweb
```

- Ver tu web. Abre tu navegador y entra en:

<http://localhost:8080>

*Si modificas tu archivo index.html en tu carpeta local, no necesitas reiniciar el contenedor — Nginx servirá automáticamente el nuevo contenido porque está usando tu carpeta directamente.*

## 7. Comunicación entre contenedores (sin -p)

```
docker network create minired
```

- Para listar las redes de Docker, el comando es muy simple:

```
docker network ls
```

| NETWORK ID   | NAME   | DRIVER | SCOPE |
|--------------|--------|--------|-------|
| f1a2b3c4d5e6 | bridge | bridge | local |
| 7g8h9i0j1k2l | host   | host   | local |
| 3m4n5o6p7q8r | none   | null   | local |

| Columna           | Descripción                                                            |
|-------------------|------------------------------------------------------------------------|
| <b>NETWORK ID</b> | Identificador único de la red                                          |
| <b>NAME</b>       | Nombre de la red (por ejemplo, bridge, host, none o una creada por ti) |
| <b>DRIVER</b>     | Tipo de red (cómo Docker gestiona las conexiones)                      |
| <b>SCOPE</b>      | Alcance (normalmente local, es decir, solo en tu máquina)              |

- Redes predeterminadas de Docker. Cuando instalas Docker, se crean 3 redes por defecto:

| Nombre | Descripción                                                               |
|--------|---------------------------------------------------------------------------|
| bridge | Red por defecto para contenedores normales (docker run sin --network)     |
| host   | El contenedor comparte la red de tu máquina host (sin aislamiento de red) |
| none   | El contenedor no tiene conexión de red                                    |

- Ver detalles de una red:  
`docker network inspect bridge`
- Crear una nueva red:  
`docker network create mi_red`
- Conectar un contenedor a una red existente:  
`docker network connect mi_red miweb`
- Desconectar un contenedor:  
`docker network disconnect mi_red miweb`

- Vamos a ver las tres redes principales: bridge, host y none, con ejemplos y casos de uso.
  - Red bridge (por defecto): la red por defecto que Docker asigna a los contenedores cuando no especificas ninguna red.
    - Cada contenedor:
      - Obtiene su propia IP interna (aislada del host).
      - Puede comunicarse con otros contenedores en la misma red bridge.
      - Puede salir a Internet a través del host (NAT).
      - No es accesible desde fuera a menos que publiques puertos (-p).

Ejemplo:

```
docker run -d --name web1 nginx
```

Este contenedor se conecta automáticamente a la red bridge.

Puedes comprobarlo con:

```
docker network inspect bridge
```

Verás algo como:

```
{
  "Containers": {
    "3a5b...": {
      "Name": "web1",
      "IPv4Address": "172.17.0.2/16"
    }
  }
}
```

Cuándo usarla: Cuando quieres aislar contenedores pero permitir que se comuniquen entre sí. En entornos de desarrollo o pequeños despliegues.

- Red host. El contenedor comparte directamente la red del host (tu máquina). No hay aislamiento de red: el contenedor usa los mismos puertos e IP que tu PC.

Ejemplo:

```
docker run -d --name web2 --network host nginx
```

Aquí, Nginx escuchará en el puerto 80 del host directamente, sin necesidad de -p.

Si abres <http://localhost>, verás el Nginx del contenedor.

Importante: No funciona en Windows con Docker Desktop (solo en Linux nativo)

Cuándo usarla: Cuando necesitas máximo rendimiento de red (sin NAT). Para herramientas de monitoreo o servicios que necesitan ver toda la red del host.

- Por qué no funciona en Windows

Docker Desktop en Windows usa una máquina virtual (VM) interna, generalmente con WSL2 o Hyper-V.

La red host en Linux significa “usar directamente la interfaz de red del sistema operativo”.

En Windows, esa “interfaz del host” corresponde a la VM de Docker, no a tu Windows directamente.

Por eso, aunque especifiques `--network host`, tu contenedor no expone los puertos directamente en Windows, y no puedes acceder con <http://localhost>.

- Qué hacer en Windows. Nos toca Mapear puertos con -p

```
docker run -d --name web1 -p 8080:80 nginx
```

Esto conecta el puerto 80 del contenedor al puerto 8080 de tu Windows, accesible desde tu navegador.

- Conclusión. En Windows + Docker Desktop, la red host no se recomienda.

- Red none. Sin red. Docker crea el contenedor sin interfaz de red.

Ejemplo:

```
docker run -it --network none ubuntu bash
```

Dentro del contenedor:

```
ifconfig
```

Solo verás:

```
lo: flags=73<UP,LOOPBACK,RUNNING> ...
```

No hay conexión a Internet ni a otros contenedores.

- Cuándo usarla: Cuando necesitas aislamiento total. Para ejecutar procesos que no deben comunicarse con nada externo, por ejemplo, scripts de análisis, pruebas de seguridad.

## Resumen comparativo

| Red    | ¿Tiene IP propia?    | ¿Accede a Internet? | ¿Accesible desde el host? | Cuándo usarla                  |
|--------|----------------------|---------------------|---------------------------|--------------------------------|
| bridge | Sí                   | Sí (vía NAT)        | Solo si publicas puertos  | Por defecto, entornos normales |
| host   | No (usa la del host) | Sí                  | Directamente              | Máximo rendimiento, Linux      |
| none   | No                   | No                  | No                        | Aislamiento total              |

Vamos a crear nuestra propia red personalizada en Docker. Es una práctica muy recomendable, sobre todo cuando trabajas con varios contenedores que deben comunicarse entre sí (por ejemplo: una web + una base de datos).

- Crear una red personalizada

```
docker network create mi_red
```

Crea una nueva red de tipo bridge (por defecto), completamente aislada de la red bridge estándar que Docker trae.

Puedes verificarla con:

```
docker network ls
```

Verás algo como:

| NETWORK ID   | NAME   | DRIVER | SCOPE |
|--------------|--------|--------|-------|
| f1a2b3c4d5e6 | bridge | bridge | local |
| 7g8h9i0j1k2l | host   | host   | local |
| 3m4n5o6p7q8r | none   | null   | local |
| 9z0x8y7w6v5u | mi_red | bridge | local |

- Crear contenedores dentro de esa red. Ahora, ejecuta tus contenedores usando la red `mi_red`:

```
docker run -d --name miweb --network mi_red nginx:latest
docker run -d --name mibase --network mi_red mysql:latest
```

Ambos contenedores están en la misma red personalizada (`mi_red`).

Por lo tanto, pueden comunicarse entre sí por nombre de contenedor (gracias al DNS interno de Docker).

No necesitas saber la IP interna de mibase.

- Comunicación entre contenedores. Dentro de esa red:
  - Los contenedores se ven por nombre (Docker DNS interno).
  - No necesitas direcciones IP.
  - No están accesibles desde otras redes ni desde el exterior (a menos que abras puertos).

- Desde `miweb`, puedes abrir una terminal dentro del contenedor:

```
docker exec -it miweb bash
```

- Luego (si es Ubuntu/Debian dentro del contenedor):

```
apt update && apt install -y iputils-ping
ping mibase
```

- Si todo está bien, deberías ver respuestas del tipo:

```
PING mibase (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: icmp_seq=1 ttl=64 time=0.12 ms
```

- Conectar un contenedor existente. Si ya tenías un contenedor creado, puedes conectarlo a la red personalizada sin recrearlo:

```
docker network connect mi_red miweb
```

Y si luego quieres desconectarlo:

```
docker network disconnect mi_red miweb
```

- Ver detalles de la red. `docker network inspect mi_red`

Te mostrará información como:

```
Tipo de red (bridge)
Contenedores conectados
Subred y gateway asignados
DNS interno
```

- Crear una red con configuración avanzada

Puedes definir el rango de IPs o el subnet si lo deseas:

```
docker network create \
  --driver bridge \
  --subnet 172.28.0.0/16 \
  --gateway 172.28.0.1 \
  mi_red_personalizada
```

## Tabla resumen

| Comando                                                       | Qué hace                        |
|---------------------------------------------------------------|---------------------------------|
| <code>docker network create mi_red</code>                     | Crea una red personalizada      |
| <code>--network mi_red</code>                                 | Asigna un contenedor a esa red  |
| <code>docker network connect mi_red &lt;contenedor&gt;</code> | Conecta un contenedor existente |
| <code>docker network inspect mi_red</code>                    | Muestra detalles de la red      |

Vemos que hay comunicación entre ambos contenedores porque están en la misma red. Los borramos.

Ejercicio Visual: Vamos a montar un entorno MySQL + phpMyAdmin usando solo contenedores individuales, para que puedas gestionar tu base de datos visualmente desde el navegador.

### 1.- Crear una red personalizada

Para que los contenedores se comuniquen por nombre:

```
docker network create mi_red
```

### 2.- Levantar MySQL

```
docker run -d --name mibase \
  --network mi_red \
  -e MYSQL_ROOT_PASSWORD=12345 \
  mysql:latest
```

`--network mi_red` → conecta MySQL a la red interna

`MYSQL_ROOT_PASSWORD=12345` → contraseña del usuario root

3.- Levantar phpMyAdmin. phpMyAdmin es una aplicación web en PHP que permite gestionar bases de datos MySQL/MariaDB de forma visual, y normalmente se expone en el puerto 8081 cuando lo ejecutas en Docker.

```
docker run -d --name phpmyadmin \
  --network mi_red \
  -e PMA_HOST=mibase \
  -e PMA_USER=root \
  -e PMA_PASSWORD=12345 \
  -p 8081:80 \
  phpmyadmin:latest
```

--network mi\_red → phpMyAdmin se comunica con MySQL por la red interna  
PMA\_HOST=mibase → nombre del contenedor MySQL  
-p 8081:80 → accederás desde el navegador en <http://localhost:8081>

4.-Probarlo. Abre tu navegador en:

<http://localhost:8081>

Usuario: root

Contraseña: 12345

Ahora podrás ver todas tus bases de datos, crear tablas y ejecutar consultas desde la interfaz web.

Recuerda que puedes listar solo la red que acabas de crear (por ejemplo minired) y ver qué contenedores están conectados a ella con este comando:

```
docker network inspect minired
```

- Para borrar una red en Docker, se usa el comando:

```
docker network rm <nombre_de_red>
docker network rm mi_red
```

Importante:

La red no se puede borrar si hay contenedores conectados a ella, aunque estén detenidos.

Primero desconecta los contenedores o bórralos:

```
docker network disconnect mi_red <contenedor>
```

```
docker rm -f <contenedor>
```

Luego ya podrás eliminar la red sin problemas.

## 8.- Persistencia de datos

Ejemplo clásico sin volúmenes (datos efímeros): Todos los contenedores que hemos creado hasta ahora al borrarlos, hemos borrado su contenido pero que podemos hacer si queremos persistencia en los datos?

- **Bind Mount:** Un Bind Mount en Docker es un tipo de montaje que vincula directamente una carpeta o archivo del sistema host (tu máquina) con una carpeta dentro del contenedor. En palabras simples: Un Bind Mount es una “carpeta compartida” entre tu computadora y el contenedor. Persisten los datos aunque borres el contenedor. Sirve sobre todo en desarrollo web o proyectos locales, cuando quieres:  
Editar archivos desde tu PC con VS Code y ver los cambios al instante dentro del contenedor (por ejemplo, en Nginx o Apache). (Por ejemplo)

Ejemplo práctico (en tu caso con Nginx)

Supón que tienes en tu PC: `C:\Users\santi\miweb\index.html`

Y quieres que Nginx lo muestre en el navegador. Ejecutas:

```
docker run -d --name miweb -p 8080:80 -v "C:\Users\santi\miweb:/usr/share/nginx/html" nginx:latest
```

| Parte                  | Explicación                                                        |
|------------------------|--------------------------------------------------------------------|
| -v                     | Monta un volumen o carpeta                                         |
| "C:\Users\santi\miweb" | Carpeta en tu máquina Windows                                      |
| /usr/share/nginx/html  | Carpeta dentro del contenedor Nginx (donde busca los archivos web) |
| nginx:latest           | Imagen oficial de Nginx                                            |

Si borras tu carpeta local los datos no persisten

- **Volumen en Docker:** Un volumen Docker es una carpeta especial gestionada por Docker que se usa para guardar datos de forma persistente, incluso cuando el contenedor se borra o se recrea. En pocas palabras: los volúmenes son la forma “oficial” de Docker de guardar datos de forma segura y duradera.

Docker los guarda dentro de su propio sistema, normalmente en:

`/var/lib/docker/volumes/`

En Windows (con Docker Desktop), eso está dentro del entorno de Docker, no directamente visible en tu Explorador de archivos.

Existen porque los contenedores son efímeros

— si eliminas un contenedor, su sistema de archivos desaparece.

Pero los volúmenes permiten separar los datos del contenedor, de modo que los datos sobreviven aunque el contenedor se destruya o se actualice.

- **Ver y gestionar volúmenes**  
Listar todos los volúmenes:  
`docker volume ls`  
Ver detalles de uno:  
`docker volume inspect datos_mysql`  
Eliminar un volumen (borra los datos):  
`docker volume rm datos_mysql`

Diferencia con un volumen Docker:

| Tipo           | Dónde se guardan los datos                   | Persiste si borras la carpeta local | Ideal para                  |
|----------------|----------------------------------------------|-------------------------------------|-----------------------------|
| Bind Mount     | En tu disco local (ej. C:\Users\santi\miweb) | No                                  | Desarrollo                  |
| Volumen Docker | En el sistema interno de Docker              | Sí                                  | Producción o bases de datos |

Un volumen Docker es un espacio de almacenamiento independiente del contenedor, gestionado por Docker, que mantiene los datos persistentes aunque borres o recrees el contenedor.

Ejemplo: Usaremos una base de datos MySQL (porque es el caso más común).

- Crear un volumen: Primero creamos un volumen vacío donde se guardarán los datos de MySQL:

```
docker volume create datos_mysql_asir
```

- Puedes comprobarlo con:

```
docker volume ls
```

```
DRIVER      VOLUME NAME
local       datos_mysql
```

- Crear el contenedor MySQL con ese volumen

```
docker run -d \
  --name mibase \
  -e MYSQL_ROOT_PASSWORD=12345 \
  -v datos_mysql:/var/lib/mysql \
  mysql:latest
```

/var/lib/mysql → es donde MySQL guarda sus datos dentro del contenedor.

datos\_mysql → el volumen donde Docker guardará esa información por fuera del contenedor.

- Crear una base de datos de prueba, Abre una terminal dentro del contenedor:

```
docker exec -it mibase bash
```

Una vez dentro:

```
mysql -u root -p
(pon la contraseña 12345)
```

Ahora crea una base de datos:

```
CREATE DATABASE prueba_asir_bd;
SHOW DATABASES;
```

Deberías ver algo así:

```
information_schema
mysql
performance_schema
prueba_asir_db
```

Sal de MySQL (exit) y del contenedor (exit otra vez).

- Borrar el contenedor completamente

```
docker rm -f mibase
```

Esto elimina el contenedor por completo, pero el volumen datos\_mysql sigue existiendo.

Verificalo con:

```
docker volume ls
```

- Crear un nuevo contenedor MySQL con el mismo volumen

```
docker run -d \
  --name mibase2 \
  -e MYSQL_ROOT_PASSWORD=12345 \
  -v datos_mysql:/var/lib/mysql \
  mysql:latest
```

- Verifica que los datos siguen ahí. Entra al nuevo contenedor:

```
docker exec -it mibase2 bash
```

Abre MySQL:

```
mysql -u root -p
```

Y ejecuta:

```
SHOW DATABASES;
```

Verás que la base de datos prueba sigue ahí, aunque el contenedor original fue eliminado.

| Acción                                      | Resultado                                 |
|---------------------------------------------|-------------------------------------------|
| Borrar contenedor                           | Se elimina, pero el volumen sigue intacto |
| Crear nuevo contenedor con el mismo volumen | Recupera los datos automáticamente        |
| Borrar el volumen                           | Los datos se pierden definitivamente      |

Los volúmenes Docker son ideales para guardar datos persistentes, como bases de datos, porque sobreviven aunque borres o actualices los contenedores.

# Mini Tutorial Dockerfile (Completo y Explicativo)

## 1. ¿Qué es un Dockerfile?

Un Dockerfile es un archivo de texto que contiene instrucciones para construir una imagen Docker personalizada.

Cada instrucción crea una capa en la imagen.

Una vez creada, esta imagen se puede ejecutar tantas veces como quieras, generando contenedores independientes pero idénticos.

Ventaja: reproducibilidad y control total sobre el entorno de ejecución.

**Idea clave:** Configuras la imagen **con todo lo que tu aplicación necesita** (sistema base, dependencias, código, configuraciones), y luego puedes crear múltiples contenedores idénticos desde esa imagen.

## 2. Sintaxis básica de un Dockerfile

```
# Imagen base
FROM ubuntu:22.04

# Autor / metadata (opcional)
LABEL maintainer="tuemail@ejemplo.com"

# Ejecutar comandos durante la construcción
RUN apt update && apt install -y curl python3 python3-pip

# Copiar archivos desde la máquina al contenedor
COPY ./app /app

# Definir directorio de trabajo
WORKDIR /app

# Comando que se ejecuta al iniciar el contenedor
CMD ["python3", "app.py"]

# Puerto expuesto (solo informativo)
EXPOSE 5000
```

### Explicación línea por línea

**FROM ubuntu:22.04**

Indica la imagen base sobre la que se construye tu contenedor. En este caso, se usa Ubuntu 22.04, así que tu contenedor tendrá todo el sistema operativo Ubuntu.

**LABEL maintainer="tuemail@ejemplo.com"**

Es información opcional sobre el autor o mantenedor de la imagen. Útil para documentación interna o si compartes la imagen.

**RUN apt update && apt install -y curl python3 python3-pip**

Ejecuta comandos durante la construcción de la imagen.

apt update → actualiza los repositorios de Ubuntu

apt install -y curl python3 python3-pip → instala curl, Python 3 y pip

Cada RUN crea una capa nueva en la imagen. (\*)

**COPY ./app /app**

Copia archivos de tu máquina (./app) al contenedor (/app). Es decir, si en tu proyecto tienes un directorio app con app.py dentro, quedará dentro del contenedor en /app.

**WORKDIR /app**

Define el directorio de trabajo dentro del contenedor. Todos los comandos posteriores (RUN, CMD) se ejecutan desde /app.

**CMD ["python3", "app.py"]**

Es el comando que se ejecuta por defecto cuando inicias un contenedor con esta imagen. Aquí ejecuta tu aplicación Python (app.py) con Python 3.

**EXPOSE 5000**

Indica el puerto que la aplicación usará (solo como información). No abre el puerto automáticamente; para acceder desde fuera necesitas mapearlo con -p 5000:5000 al hacer docker run.

**Resumen conceptual**

Se toma Ubuntu como base.

Se instalan dependencias necesarias (curl, Python, pip).

Se copian tus archivos al contenedor.

Se define /app como directorio de trabajo.

Se indica el comando por defecto (python3 app.py).

Se expone un puerto para la app (informativo).

(\*) Cada línea del Dockerfile crea una capa en la imagen, por eso es buena práctica combinar comandos RUN para no generar demasiadas capas innecesarias.

Qué significa que cada línea crea una capa. Cuando construyes una imagen con Docker:

```
RUN apt update
```

```
RUN apt install -y python3 python3-pip
```

Docker crea una capa nueva por cada línea que cambia el sistema de archivos (por ejemplo, RUN, COPY, ADD). Cada capa es como un “snapshot” parcial de la imagen en ese punto.

Cómo funcionan las capas. Cada capa se apila sobre la anterior. Las capas se reutilizan si no cambian, lo que hace que las imágenes sean más rápidas de construir y más eficientes en espacio. Pero muchas capas pequeñas pueden hacer que la imagen final sea más grande de lo necesario.

Por qué se recomienda combinar comandos RUN. En lugar de esto:

```
RUN apt update
```

```
RUN apt install -y python3
```

```
RUN apt install -y python3-pip
```

```
RUN apt install -y curl
```

Que crea 4 capas para la instalación, lo ideal es combinarlas en una sola línea:

```
RUN apt update && apt install -y python3 python3-pip curl
```

Menos capas → imagen más ligera. Más rápido de construir. Mantiene la imagen más limpia y ordenada

COPY ./app /app crea otra capa (porque agrega archivos). WORKDIR /app también crea una capa (aunque muy pequeña, solo cambia el contexto de trabajo).

No siempre se combinan estas líneas porque tienen funciones distintas, pero para RUN sí es buena práctica unir lo que se pueda.

## Resumen visual



Cada flecha representa una capa apilada sobre la anterior. Cuantas menos capas innecesarias tengas, más eficiente será la imagen.

## 3. Instrucciones más importantes

| Instrucción | Qué hace                                                                              |
|-------------|---------------------------------------------------------------------------------------|
| FROM        | Imagen base sobre la que se construye tu imagen (Ubuntu, Debian, Alpine, etc.)        |
| LABEL       | Metadatos del contenedor                                                              |
| RUN         | Ejecuta comandos durante la construcción (instalar dependencias, actualizar paquetes) |
| COPY        | Copia archivos de tu máquina al contenedor                                            |
| WORKDIR     | Define directorio de trabajo dentro del contenedor                                    |
| ENV         | Define variables de entorno                                                           |
| EXPOSE      | Puerto que el contenedor usará (solo informativo)                                     |
| CMD         | Comando por defecto al ejecutar el contenedor                                         |
| ENTRYPOINT  | Define comando fijo, opcionalmente recibe argumentos de CMD                           |
| USER        | Define el usuario con el que correrá el contenedor                                    |

---

## 4.-Construir una imagen desde Dockerfile

```
docker build -t miimagen:1.0
```

-t miimagen:1.0 → nombre y tag de la imagen  
→ contexto de construcción (carpeta actual)

Se obtiene una imagen que **tiene todo lo necesario** (sistema, Python, librerías, código) y sobre la que se pueden crear múltiples contenedores reproducibles.

```
docker image ls
```

## 5. Ejecutar un contenedor desde la imagen

```
docker run -d --name micontenedor -p 8080:5000 miimagen:1.0 .
```

d → ejecución en segundo plano

-p host:contenedor → mapea el puerto para acceder desde el host

--name → nombre del contenedor

Puedes crear otros contenedores idénticos simplemente cambiando --name:

```
docker run -d --name micontenedor2 -p 8081:5000 miimagen:1.0
```

Docker mapea (conecta) un puerto de tu máquina (host) con un puerto dentro del contenedor.

8080 → es el puerto de tu ordenador (host)

5000 → es el puerto dentro del contenedor

## 6. Ejemplo práctico: Python + Flask

Vamos a realizar un ejemplo muy simple y totalmente funcional en Docker, usando Python + Flask, de modo que puedas abrirlo desde el navegador y ver el programa corriendo.

¿Qué es Flask?

Flask es un microframework de Python para crear aplicaciones web de forma muy sencilla.

En palabras simples: Flask te permite convertir tu código Python en una aplicación web que puedes abrir desde el navegador (por ejemplo, <http://localhost:8080>).

¿Qué significa “microframework”?

“Micro” no quiere decir que sea pequeño o limitado.

Significa que no trae muchas cosas preinstaladas como otros frameworks (por ejemplo, Django).

Solo incluye lo esencial:

Un servidor web integrado

Un sistema de rutas (para definir URLs como /, /sumar, etc.)

Soporte para formularios y peticiones HTTP

Tú decides si añadirle más funciones (bases de datos, plantillas, etc.).

¿Por qué se usa con Docker? Porque Flask: Se ejecuta en un solo archivo (app.py).

No necesita configuraciones complejas. Es ideal para ejemplos educativos o microservicios.

| Concepto           | Explicación                                                       |
|--------------------|-------------------------------------------------------------------|
| Flask              | Framework de Python para crear apps web fácilmente                |
| Tipo               | Microframework (ligero y flexible)                                |
| Puerto por defecto | 5000                                                              |
| Uso típico         | APIs, páginas web simples, microservicios                         |
| Ventaja            | Puedes transformar un script de Python en una web sin complicarte |

Nuestro ejemplo consiste en un programa que pida dos números por pantalla y los sume, hasta que decidas Salir.

- **Crea la estructura de tu proyecto**

En una carpeta (por ejemplo app\_suma), crea estos archivos:

```
app_suma/  
├── Dockerfile  
└── app.py
```

- **Código del programa: app.py**

Este pequeño servidor Flask mostrará un formulario web para sumar dos números:

```
from flask import Flask, request  
  
app = Flask(__name__)  
  
@app.route('/')  
def menu():  
    return '''  
        <h1>Calculadora simple</h1>  
        <form action="/sumar" method="post">  
            <label>Número 1:</label><br>  
            <input type="number" name="num1" required><br><br>  
            <label>Número 2:</label><br>  
            <input type="number" name="num2" required><br><br>  
            <button type="submit">Sumar</button>  
        </form>  
        <br>  
        <a href="/salir">Salir</a>  
    '''  
  
@app.route('/sumar', methods=['POST'])  
def sumar():  
    num1 = float(request.form['num1'])  
    num2 = float(request.form['num2'])  
    resultado = num1 + num2  
    return f'''  
        <h2>Resultado: {resultado}</h2>  
        <a href="/">Volver</a>  
    '''  
  
@app.route('/salir')  
def salir():  
    return "<h2>;Hasta luego!</h2>"  
  
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000)
```

Este programa:

- Muestra el formulario
- Suma los dos números y muestra el resultado
- Permite “salir” (solo muestra un mensaje)
- Escucha en el puerto 5000 dentro del contenedor

## Explicación del código:

- Importación de librerías

```
from flask import Flask, request
```

Flask → Es la clase principal que te permite crear la aplicación web.

request → Se usa para recoger los datos que el usuario envía desde un formulario (por ejemplo, los números que escribe).

- Crear la aplicación Flask

```
app = Flask(__name__)
```

Crea una instancia de la app web.

name\_\_ le dice a Flask cómo ubicar los archivos y recursos del proyecto. Esta variable app se usará para definir rutas (páginas o URLs) de tu sitio.

- Definir la ruta principal / (la página inicial)

```
@app.route('/')
def menu():
    return '''
        <h1>Calculadora simple</h1>
        <form action="/sumar" method="post">
            <label>Número 1:</label><br>
            <input type="number" name="num1" required><br><br>
            <label>Número 2:</label><br>
            <input type="number" name="num2" required><br><br>
            <button type="submit">Sumar</button>
        </form>
        <br>
        <a href="/salir">Salir</a>
    '''
```

@app.route('/') → Define la URL principal, o sea, la página http://localhost:8085/.

Devuelve HTML que se mostrará en el navegador.

El formulario envía los números a la ruta /sumar usando el método POST.

return "En Python, las tres comillas (") sirven para escribir cadenas de texto multilínea.

Es decir, puedes poner varias líneas de texto sin usar \n.

Así, cuando visitas http://localhost:8085/, Flask ejecuta esta función menu(), y el navegador renderiza el HTML que devuelves dentro del return.

- Ruta /sumar → realiza la suma

```
@app.route('/sumar', methods=['POST'])
def sumar():
    num1 = float(request.form['num1'])
    num2 = float(request.form['num2'])
    resultado = num1 + num2
    return f'''
        <h2>Resultado: {resultado}</h2>
        <a href="/">Volver</a>
    '''
```

Esta función se ejecuta cuando el usuario envía el formulario.  
request.form['num1'] y request.form['num2'] → obtienen los valores que el usuario escribió.  
Se convierten a float para poder sumarlos.  
Muestra el resultado en pantalla con un pequeño HTML.

- Ruta /salir

```
@app.route('/salir')
def salir():
    return "<h2>¡Hasta luego!</h2>"
```

Si haces clic en “Salir”, se muestra una página con ese mensaje.  
No cierra el programa, simplemente muestra una nueva ruta.

Cómo se cerraría el programa

Puedes detener el contenedor con:

```
docker stop mi_app_suma
```

Y si ya no lo necesitas:

```
docker rm mi_app_suma
```

Eso sí apaga completamente el programa (Flask deja de escuchar y el contenedor se detiene).

Podrías cerrarlo dentro del programa .py, pero no es recomendable.

Podrías usar algo como esto:

```
import os
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/salir')
```

```
def salir():
```

```
    os._exit(0) # Fuerza el cierre del proceso (NO recomendado)
```

Pero esto mata el servidor abruptamente, y en un entorno real (como Docker o producción) no es buena práctica, porque:

Puede interrumpir peticiones activas.

No deja limpiar recursos ni guardar logs.

En Docker, el contenedor se apaga pero puede reiniciarse automáticamente si está configurado para ello.

- Ejecutar la aplicación

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Indica que el programa debe iniciar el servidor Flask.

host='0.0.0.0' → permite que Flask acepte conexiones desde fuera del contenedor, es decir, desde tu navegador en el host.

port=5000 → puerto interno donde Flask escucha.

(Luego lo mapeas con -p 8085:5000 para abrirlo desde tu navegador en http://localhost:8085).

- **Dockerfile**

```
# Imagen base
FROM python:3.10

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiar el programa al contenedor
COPY app.py /app

# Instalar Flask
RUN pip install flask

# Exponer el puerto interno
EXPOSE 5000

# Comando para ejecutar la aplicación
CMD ["python", "app.py"]
```

- **Construir la imagen**

Desde la carpeta app\_suma, abre una terminal (PowerShell o CMD):

```
docker build -t app_suma:1.0 .
```

Esto crea una imagen llamada app\_suma:1.0

- **Ejecutar el contenedor y mapear puertos**

Ejecuta el contenedor mapeando el puerto interno 5000 al 8085 del host:

```
docker run -d --name mi_app_suma -p 8085:5000 app_suma:1.0
```

-d → modo segundo plano

--name → nombre del contenedor

-p 8085:5000 → conecta el puerto 8085 del host con el 5000 del contenedor

- **Prueba desde tu navegador**

Abre en tu navegador:

<http://localhost:8080>

- **(Opcional) Detener y eliminar**

```
docker stop mi_app_suma
```

```
docker rm mi_app_suma / docker rmi app_suma:1.0
```

## 7. Bind mount en desarrollo

```
docker run -d -p 8085:5000 -v "C:\Users\santi\micarpeta:/app/src flaskapp:1.0
```

Permite ver cambios en tiempo real desde tu máquina en el contenedor.

Al tener este bind mount, si editas app.py o los archivos HTML en tu máquina (con VS Code, por ejemplo),

el contenedor verá los cambios al instante

Así no necesitas reconstruir la imagen cada vez que cambias una línea de código.

Solo recomendable en desarrollo.

-d: ejecuta el contenedor en segundo plano (modo “detached”)

-p 8085:5000: expone el puerto 5000 del contenedor al puerto 8085 de tu máquina  
→ así accedes en tu navegador a <http://localhost:8085>

-v "C:\Users\santi\micarpeta:/app/src": monta una carpeta local (Bind Mount)  
→ todo lo que haya en tu PC dentro de C:\Users\santi\micarpeta se verá dentro del contenedor en /app/src

flaskapp:1.0: es la imagen que estás ejecutando

## 8. Buenas prácticas en Dockerfile

Usar imágenes ligeras (slim, alpine).

Minimizar capas combinando comandos RUN.

Evitar instalar dependencias innecesarias.

Usar .dockerignore para no copiar archivos que no hacen falta.

Configurar variables de entorno con ENV.

Evitar ejecutar contenedores como root.

Ordenar instrucciones para aprovechar cache y acelerar la construcción.

## 9. Inspección y logs

Ver capas y tamaño de la imagen:

```
docker history miimagen:1.0
```

Ver información de un contenedor:

```
docker inspect micontenedor
```

Ver logs del contenedor:

```
docker logs micontenedor
```

## 10. Resumen visual

| Concepto            | Ejemplo / Explicación                                               |
|---------------------|---------------------------------------------------------------------|
| Imagen base         | FROM python:3.11-slim (Debian Slim + Python)                        |
| Dependencias        | RUN pip install -r requirements.txt                                 |
| Copiar código       | COPY ./src ./src                                                    |
| Directorio trabajo  | WORKDIR /app                                                        |
| Comando por defecto | CMD ["python", "src/app.py"]                                        |
| Puerto              | EXPOSE 5000                                                         |
| Construir imagen    | docker build -t flaskapp:1.0 .                                      |
| Ejecutar contenedor | docker run -d -p 5000:5000 flaskapp:1.0                             |
| Reproducibilidad    | Puedes crear múltiples contenedores idénticos desde la misma imagen |

(Continúa – Docker Compose y Docker Desktop con extensión Portainer)